Ch 1. Terraform 소개

테라폼은 HashiCorp사에서 개발한 **IaC (Infrastructure as Code, 코드로써의 인프라)** 도구입니다. 인프라를 수동으로 구성하는 대신, **선언적인 구성 파일**을 작성하여 인프라를 프로비저닝하고 관리할 수 있게 해줍니다.

1. 테라폼의 핵심 역할

IaC의 구현

- 인프라를 코드로 정의하고 관리하여, **반복 가능성**, **일관성**, **버전 관리**를 가능하게 합니다.
- 인프라의 **현재 상태(State)**를 파일로 관리하며, 이 상태와 작성된 코드를 비교하여 변경 사항만 적용합니다.

선언적 방식 (Declarative)

- "어떻게" 인프라를 만들어야 하는지 대신, "최종적으로 **어떤 상태**가 되어야 하는지"를 명시합니다.
- 테라폼이 현재 상태를 파악하고, 목표 상태에 도달하기 위한 **필요한 실행 계획(Plan)**을 스스로 생성합니다.

멀티 클라우드 지원 (Multi-Cloud Support)

- 특정 클라우드에 종속되지 않고, **AWS, Azure, Google Cloud Platform (GCP)** 등 주요 클라우드뿐만 아니라 **Kubernetes, VMWare** 등 다양한 플랫폼의 인프라를 하나의 도구로 관리할 수 있습니다.
- 이는 **프로바이더(Provider)**라는 플러그인 시스템을 통해 가능합니다.

---

2. 테라폼의 주요 특징 및 구성 요소

A. HCL (HashiCorp Configuration Language)

- 테라폼 구성 파일을 작성하는 데 사용되는 언어입니다.
- 사람이 읽기 쉬우면서도 기계가 해석하기 용이하도록 설계되었습니다. (JSON 포맷도 지원)

B. State 파일

- 테라폼이 관리하는 **실제 인프라의 현재 상태**를 기록한 파일 (`terraform.tfstate`)입니다.
- 테라폼은 이 파일과 `.tf` 구성 파일을 비교하여 어떤 변경이 필요한지 판단합니다.
- 협업 환경에서는 S3나 Azure Storage와 같은 **원격 백엔드**에 저장하여 관리합니다.

C. Execution Plan (실행 계획)

- `terraform plan` 명령어를 실행하면 테라폼은 **현재 인프라 상태**와 **구성 파일의 목표 상태**를 비교하여 어떤 리소스를 생성, 수정, 또는 삭제할지 미리 보여줍니다.
- 이는 배포 전에 잠재적인 위험을 검토할 수 있게 해주는 **매우 중요한 안전 장치**입니다.

D. 모듈 (Modules)

- 재사용 가능한 테라폼 코드의 컨테이너입니다.
- 특정 인프라 셋업 (예: 표준 웹 서버 구성)을 모듈로 만들어 여러 프로젝트에서 쉽게 재활용하고 일관성을 유지할 수 있게 해줍니다.

---

3. 테라폼 사용 워크플로우

테라폼을 사용하여 인프라를 배포하는 표준 과정은 다음과 같습니다.

1. `terraform init`: 작업 디렉토리를 초기화하고, 필요한 **프로바이더**와 **모듈**을 다운로드합니다.
2. `terraform plan`: 구성 파일을 기반으로 **실행 계획**을 생성하고 변경 사항을 미리 검토합니다.
3. `terraform apply`: 사용자 승인을 받은 후, 실행 계획에 따라 **실제 인프라를 프로비저닝**합니다.
4. `terraform destroy`: 배포된 인프라 리소스를 모두 **제거**합니다. (신중하게 사용해야 합니다.)

테라폼은 복잡한 인프라를 안전하고 효율적으로 관리하는 데 필수적인 도구입니다.

---

4. 테라폼 vs. 다른 IaC 도구 비교

테라폼은 **프로비저닝 도구**에 속하며, 주로 **구성 관리 도구**인 Ansible이나 **클라우드 고유 도구**인 CloudFormation과는 역할과 접근 방식에서 차이가 있습니다.

1. 테라폼 (Terraform)

특징	설명
분류	**프로비저닝 도구** (Provisioning Tool)
주요 역할	인프라 생성, 수정, 삭제 (IaaS 계층)
접근 방식	**선언적** (Declarative): 최종 상태 정의
지원 범위	**멀티 클라우드** (AWS, Azure, GCP 등)
언어	**HCL** (HashiCorp Configuration Language)
핵심 요소	**State File**을 통해 인프라 상태 추적

2. AWS CloudFormation

특징	설명
분류	**프로비저닝 도구** (Provisioning Tool)
주요 역할	인프라 생성, 수정, 삭제
접근 방식	**선언적** (Declarative)
지원 범위	**AWS 전용**
언어	YAML 또는 JSON
핵심 요소	**Stack** 단위로 리소스 관리

3. Ansible

특징	설명
분류	**구성 관리 도구** (Configuration Management Tool)
주요 역할	프로비저닝된 인프라 **내부** 설정 (소프트웨어 설치, 파일 설정 등)
접근 방식	**절차적** (Procedural/Imperative): 실행할 단계 정의
지원 범위	광범위 (OS, 소프트웨어)
언어	YAML (Playbook 작성)
핵심 요소	**Agentless** (별도 클라이언트 설치 불필요)

📋 주요 IaC 도구 비교 요약

도구	주된 목적	접근 방식	멀티 클라우드 지원	장점
테라폼	**인프라 프로비저닝** (Provisioning)	선언적 (Declarative)	**O** (최고 수준)	이기종 클라우드 관리, State 기반 정확한 추적

Ansible	**구성 관리** (Configuration Management)	절차적 (Procedural)	O (인프라 생성보다는 구성에 초점)	Agentless, 쉬운 문법, 소프트웨어 설정에 강력
CloudFormation	**인프라 프로비저닝**	선언적 (Declarative)	X (AWS 전용)	AWS 서비스에 대한 가장 빠른 업데이트 및 지원

> 🔥 통합 사용 예시:
>
> - **테라폼**으로 AWS **EC2 인스턴스**와 **네트워크**를 **프로비저닝**합니다.
> - 프로비저닝된 EC2 인스턴스에 **Ansible**을 사용하여 **웹 서버 소프트웨어** (Apache/Nginx)를 설치하고 **최초 설정**을 구성합니다.
>
> 이처럼 두 도구는 상호 보완적으로 사용되어 **인프라 생성과 설정**의 전체 라이프사이클을 자동화할 수 있습니다.